

LAPORAN PRAKTIKUM SISTEM OPERASI

MODUL 11 MEMORI XINU



Disusun Oleh :

Ilham Lii Assidaq 2311104068

Dosen Pengampu :

Yudha Islami Sulistya, S.Kom., M.Cs

PROGRAM STUDI S1 SOFTWARE ENGINEERING

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

1. Dasar Teori

A. Struktur Memori dan Manajemen Memori Xinu

- Karakteristik Utama: Xinu tidak menggunakan Virtual Memory atau Paging seperti Linux, melainkan menggunakan manajemen memori fisik langsung berbasis struktur Linked List (memlist).
- 4 Bagian Utama Memory Image:
 1. Text Segment: Memuat seluruh kode program hasil kompilasi (fungsi, utilitas sistem, perintah shell) pada alamat memori terendah.
 2. Data Segment: Menyimpan variabel global yang sudah diinisialisasi atau diberi nilai awal.
 3. BSS Segment: Menyimpan variabel global tanpa inisialisasi (otomatis bernilai 0).
 4. Free Space: Sisa ruang kosong tempat dialokasikannya stack proses dan heap.

B. Pembagian Free Space Saat Runtime

- Stack:
 1. Dibuat otomatis saat fungsi create() dipanggil dan dihapus ketika proses selesai.
 2. Tumbuh dari alamat memori tertinggi ke bawah.
 3. Berfungsi menampung variabel lokal, return address, dan konteks proses.
- Heap:
 1. Dialokasikan secara dinamis dari bawah ke atas menggunakan perintah getmem().
 2. Harus dihapus manual menggunakan freemem(). Jika lupa, akan memicu kebocoran memori (memory leak).

C. 4 Fungsi Inti Manajemen Memori Xinu

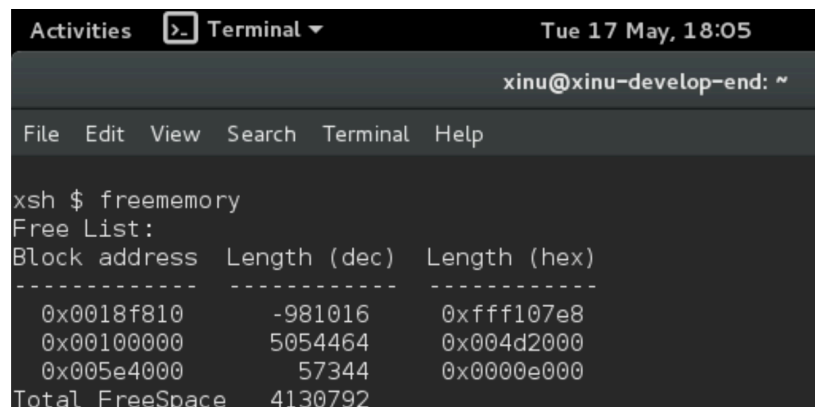
- getstk(size): Digunakan oleh kernel untuk mengalokasikan ruang stack baru dari memori paling atas.
- freestk(stackaddr, size): Digunakan untuk menghapus dan membebaskan ruang stack setelah proses berakhir.
- getmem(nbytes): Mengambil memori secara dinamis dari heap. Mengembalikan nilai SYSERR jika memori penuh.
- freemem(ptr, size): Mengembalikan memori heap yang telah dipakai ke free space untuk mencegah memory leak.

2. Manfaat

Melalui praktikum pada modul ini, manfaat yang saya dapatkan adalah mampu memahami arsitektur manajemen memori fisik secara langsung melalui struktur data Linked List (memlist) tanpa kompleksitas Virtual Memory. Saya juga dapat memahami pembagian ruang memori free space menjadi stack yang tumbuh ke bawah untuk konteks proses dan heap yang tumbuh ke atas untuk alokasi dinamis. Selain itu, pembelajaran mengenai fungsi-fungsi inti seperti `getmem()` dan `freemem()` secara nyata meningkatkan keterampilan praktis saya dalam mengelola memori sistem secara efisien sekaligus melatih kepekaan logika saya untuk mencegah terjadinya kebocoran memori (memory leak).

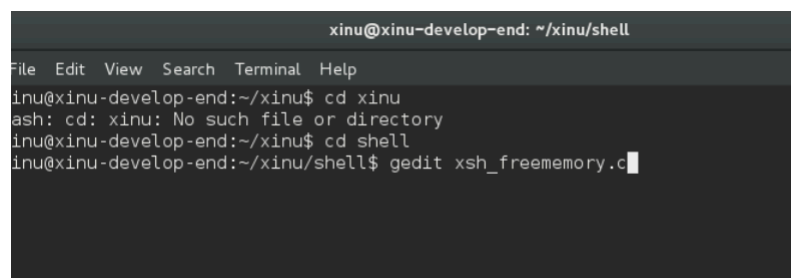
3. Jurnal

- A. [80 poin] Buatlah perintah baru bernama `freememory` yang memiliki dua fungsi berikut:
- [40 poin] Menampilkan seluruh free memory block yang tercatat dalam free memory list pada Xinu.
 - [40 poin] Menghitung dan menampilkan total ukuran free memory berdasarkan seluruh block yang ada pada list tersebut.



```
Activities [Terminal] Tue 17 May, 18:05
xinu@xinu-develop-end: ~
File Edit View Search Terminal Help

xsh $ freememory
Free List:
Block address  Length (dec)  Length (hex)
-----
0x0018f810     -981016      0xffff107e8
0x00100000      5054464      0x004d2000
0x005e4000       57344       0x0000e000
Total FreeSpace 4130792
```



```
xinu@xinu-develop-end: ~/xinu/shell
File Edit View Search Terminal Help

xinu@xinu-develop-end:~/xinu$ cd xinu
ash: cd: xinu: No such file or directory
xinu@xinu-develop-end:~/xinu$ cd shell
xinu@xinu-develop-end:~/xinu/shell$ gedit xsh_freememory.c
```

```

xsh_freememory.c
~/xinu/shell

shellcmd xsh_freememory(int nargs, char *args[]) {

    struct memblk *next; /* pointer to traverse free list */
    uint32 totalFreeSpace; /* accumulator for total free memory */

    totalFreeSpace = 0;

    /* Print header */
    printf("Free List:\n");
    printf("Block address Length (dec) Length (hex)\n");
    printf("-----\n");

    /* Traverse the free memory linked list */
    for (next = memlist.mnext; next != NULL; next = next->mnext) {
        printf("0x%08x %12d 0x%08x\n",
            (uint32)next,
            (int32)next->mlength,
            next->mlength);

        /* Accumulate total free space */
        totalFreeSpace += next->mlength;
    }

    /* Print total free space */
    printf("Total FreeSpace %d\n", totalFreeSpace);
    return 0;
}

```

C Tab Width: 8 Ln 30, Col 9 IN

```

shprototypes.h
~/xinu/include

extern shellcmd xsh_steep (int32, char *[]);

/* in file xsh_udpdump.c */
extern shellcmd xsh_udpdump (int32, char *[]);

/* in file xsh_udpecho.c */
extern shellcmd xsh_udpecho (int32, char *[]);

/* in file xsh_udpserver.c */
extern shellcmd xsh_udpserver (int32, char *[]);

/* in file xsh_uptime.c */
extern shellcmd xsh_uptime (int32, char *[]);

/* in file xsh_mycmd.c */
extern shellcmd xsh_mycmd (int32, char *[]);

/* in file xsh_namecmd.c */
extern shellcmd xsh_namecmd (int32, char *[]);

/* in file xsh_freememory.c */
extern shellcmd xsh_freememory (int32, char *[]);

/* in file xsh_help.c */
extern shellcmd xsh_help (int32, char *[]);

/* in file xsh_tee.c */
extern shellcmd xsh_tee (int32, char *[]);

C/C++/ObjC Header Tab Width: 8 Ln 85, Col 1 INC

```

```

File Edit View Search Terminal Help
-00 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=""`cat version`"-I.
nclude -o binaries/xsh_udpecho.o ../shell/xsh_udpecho.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wal
-00 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=""`cat version`"-I.
nclude -o binaries/xsh_udpserver.o ../shell/xsh_udpserver.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wal
-00 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=""`cat version`"-I.
nclude -o binaries/xsh_uptime.o ../shell/xsh_uptime.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wal
-00 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=""`cat version`"-I.
nclude -o binaries/clkdsp.o ../system/clkdsp.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wal
-00 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=""`cat version`"-I.
nclude -o binaries/ctxsw.o ../system/ctxsw.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wal
-00 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=""`cat version`"-I.
nclude -o binaries/intr.o ../system/intr.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wal
-00 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=""`cat version`"-I.
nclude -o binaries/ttydispatch.o ../device/tty/ttydispatch.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wal
-00 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=""`cat version`"-I.
nclude -o binaries/ethdispatch.o ../device/eth/ethdispatch.S

Loading object files to produce GRUB bootable xinu
Copying xinu.elf to xinu.boot in the TFTP directory.
xinu@xinu-develop-end:~/xinu/compile$

```

```
Welcome to Xinu!

xsh $ freememory
Free List:
Block address Length (dec) Length (hex)
-----
0x0018e110      -975128  0xffff11ee8
0x00100000      5070848  0x004d6000
0x005e8000        57344  0x0000e000
Total FreeSpace 4153064
xsh $
```

```
CTRL-A Z for help | 115200 8N1 | NOR | Minicom 2.7 | VT102 | Online 0:1 |
```

B. [4 poin per subsoal] Jawablah pertanyaan berikut:

a. Mengapa Xinu memisahkan data segment dan BSS segment?

Pemisahan ini dilakukan demi efisiensi ukuran berkas image kernel, mempercepat proses booting saat inisialisasi memori, serta mempermudah pengaturan hak akses area memori sistem.

b. Bagaimana alokasi dan dealokasi memori selama eksekusi memengaruhi ukuran free space?

- Alokasi (getmem/create): Memotong memori dari free space, sehingga ukurannya berkurang.
- Dealokasi (freemem/freestk): Mengembalikan memori ke sistem, sehingga ukuran free space bertambah kembali.

c. Mengapa penggunaan heap lebih berpotensi menimbulkan masalah dibandingkan stack?

Heap lebih berpotensi menimbulkan masalah karena alokasi dan dealokasinya harus diatur secara manual oleh programmer (getmem() dan freemem()). Jika lupa dibebaskan, akan terjadi kebocoran memori (memory leak), dan jika salah mengelola pointer, dapat menyebabkan corrupted data atau crash.

Sementara itu, stack dikelola secara otomatis oleh sistem operasi (dialokasikan saat proses dibuat dan otomatis dihapus saat proses berakhir), sehingga jauh lebih aman dari risiko kelalaian manusia.

d. Mengapa Xinu menggunakan struktur linked list untuk menyimpan free block?

Linked list digunakan karena strukturnya sangat fleksibel untuk menangani ukuran blok memori yang dinamis.

Saat terjadi alokasi atau dealokasi, kernel Xinu dapat dengan mudah menambah, menghapus, atau menggabungkan (coalescing) blok-blok memori bebas tanpa perlu menggeser data lain di memori fisik.

e. Apa tantangan utama dalam penggunaan heap di Xinu?

Kebocoran Memori (Memory Leak): Terjadi jika memori yang telah dialokasikan dengan `getmem()` lupa dikembalikan menggunakan `freemem()`, sehingga memori bebas akan terus berkurang.